# Project 2: LiDAR-Based SLAM

Junyi Xu
Email: jux022@ucsd.edu

## I. Introduction

This project aims to implement a 2D Simultaneous Localization and Mapping (SLAM) pipeline for a differential-drive robot using wheel encoders, IMU, LiDAR, and RGB-D measurements. Accurate SLAM is crucial for autonomous navigation in unknown environments without external reference systems.

My approach sequentially integrates these sensors: estimating dead-reckoning odometry from encoders and IMU, correcting local drift via regularized Point-to-Line ICP scan matching, and constructing 2D occupancy and colored texture maps. Finally, I eliminate global cumulative drift using Pose Graph Optimization (GTSAM library) with proximity-based loop closures.

## II. Problem Formulation

I formulate the SLAM problem as estimating the joint posterior probability of the robot's trajectory $X$ and the map $M$ given the history of sensor measurements $Z$ and control inputs $U$:

$$P(X_{1:T}, M \mid Z_{1:T}, U_{1:T}) \tag{1}$$

**State Space:** At any discrete time step $t$, the robot's pose is defined in the $SE(2)$ space relative to the world frame as $\mathbf{x}_t = [x_t, y_t, \theta_t]^T \in \mathbb{R}^2 \times SO(2)$.

**Control Input:** The pseudo-control input $\mathbf{u}_t = [v_t, \omega_t]^T$ consists of the linear velocity $v_t$ derived from encoders and the angular velocity $\omega_t$ from the IMU.

**Observations:** The measurements at time $t$ include the LiDAR scan $Z_t^{\text{lidar}} = \{\mathbf{p}_{t,1}, \ldots, \mathbf{p}_{t,K}\}$ and the Kinect RGB-D image pairs $Z_t^{\text{cam}} = \{I_t^{\text{rgb}}, I_t^{\text{disp}}\}$.

**Map Representation:** The environment is modeled as an Occupancy Grid Map $M = \{m_i\}$, where each grid cell $m_i$ holds a probability $P(m_i = 1)$ representing the belief that the physical space is occupied.

## III. Technical Approach

### A. Coordinate Frames and Sensor Calibration

Establishing the rigid body transformations between sensors is critical. The Hokuyo LiDAR is offset along the forward x-axis by $\mathbf{t}_{L\to B}$. A 2D point $\mathbf{p}^L$ in the LiDAR frame is transformed to the world frame (W) via:

$$\mathbf{p}^W = R(\theta_t)(\mathbf{p}^L + \mathbf{t}_{L\to B}) + \begin{bmatrix} x_t \\ y_t \end{bmatrix} \tag{2}$$

Similarly, the Kinect camera is located at $(0.18, 0.005, 0.36)$ m with respect to the robot center and has an orientation with roll 0 rad, pitch 0.36 rad, and yaw 0.021 rad. The intrinsic parameters of the depth camera are defined as:

$$K = \begin{bmatrix} f_{s_u} & f_{s_\theta} & c_u \\ 0 & f_{s_v} & c_v \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 585.05 & 0 & 242.94 \\ 0 & 585.05 & 315.84 \\ 0 & 0 & 1 \end{bmatrix} \tag{3}$$

### B. Odometry Model (Dead Reckoning)

The initial pose is propagated using a differential-drive kinematic model. Given encoder counts $[FR, FL, RR, RL]$ and 0.0022 meters per tick, the traveled distance of the right ($d_R$) and left ($d_L$) wheels are:

$$d_R = \frac{FR + RR}{2} \times 0.0022, \quad d_L = \frac{FL + RL}{2} \times 0.0022 \tag{4}$$

The linear velocity is $v_t = (d_R + d_L)/(2\Delta t)$. Coupled with the IMU yaw rate $\omega_t$, the pose updates as:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \begin{bmatrix} v_t \cos\theta_t \\ v_t \sin\theta_t \\ \omega_t \end{bmatrix} \Delta t \tag{5}$$

### C. Point-to-Line ICP Scan Matching

Given two consecutive LiDAR scans, I use ICP to estimate the 2D relative pose change ${}_tT_{t+1}$. Once the relative pose change is known, the robot pose $T_{t+1}$ at time $t+1$ can be approximated as:

$$T_{t+1} = T_t * {}_tT_{t+1} \tag{6}$$

To handle flat walls efficiently, a Point-to-Line error metric is adopted. The optimal transformation $\Delta\mathbf{x}$ is solved by minimizing the point-to-line residual regularized by the odometry prior to prevent corridor degradation:

$$\Delta\mathbf{x} = \arg\min_{\Delta\mathbf{x}} \sum_i \|J_i \Delta\mathbf{x} - r_i\|^2 + \lambda \|\Delta\mathbf{x} - \Delta\mathbf{x}_{\text{odom}}\|^2 \tag{7}$$

### D. Occupancy and Texture Mapping

The 2D occupancy grid is updated using the log-odds formulation. For each valid LiDAR ray, Bresenham's line algorithm traces cells from the LiDAR origin to the hit point.

For texture mapping, there is an x-axis offset between the depth and RGB cameras. Given the values $d$ at pixel $(i, j)$ of the disparity image, I use the exact following conversions to obtain the physical depth and the pixel location $(rgbi, rgbj)$ of the associated RGB color:

$$dd = (-0.00304d + 3.31) \tag{8}$$

$$\text{depth} = \frac{1.03}{dd} \tag{9}$$

$$rgbi = (526.37i + 19276 - 7877.07dd)/585.051 \tag{10}$$

$$rgbj = (526.37j + 16662)/585.051 \tag{11}$$

Valid pixels are projected into the 3D world frame, thresholded by height (Z $\approx$ 0) to extract the floor plane, and color-mapped based on the synchronized RGB timestamps.

### *E. Pose Graph Optimization (Loop Closure)*

GTSAM is utilized to eliminate cumulative errors. A factor graph is constructed with $SE(2)$ pose nodes, where odometry edges constrain sequential poses. To identify loop closures, I combine two complementary strategies: **fixed-interval** detection (proposing candidate edges every 10 poses) and **proximity-based** detection (querying non-sequential poses within a $0.5$m radius). Finally, a strict **consistency check** is applied to all candidates: loop closure edges are added to the graph only if the ICP fitness score (MSE) is below $0.0015$ and the relative translational displacement ensures physical plausibility ($\Delta x, \Delta y < 0.5$m).

## IV. Results and Discussion

### *A. Warm-up: 3D ICP Registration*

Prior to 2D scan matching, I validated my ICP implementation via 3D point cloud registration. By discretizing yaw-angle initializations to prevent converging to poor local optima, and optimizing the 6-DoF transformation via SVD, shown in Fig. 1, the source point and target point align well. This confirms the foundational correctness of the ICP in this project.

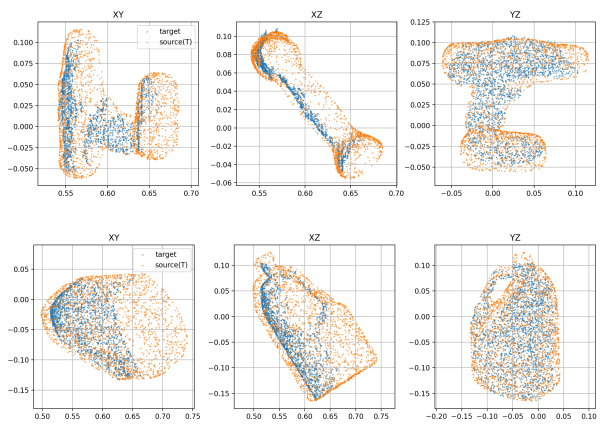


Fig. 1. 3D ICP Warm-up point-cloud registration. Top: Drill. Bottom: Liquid container. Target points and transformed source points match perfectly.

### *B. Trajectory Evolution and Fitness*

Seq = 20: In Fig. 2, dead-reckoning already shows noticeable global drift as the integration error accumulates along the long hallway, leading to a slightly bent corridor structure. After applying my regularized Point-to-Line ICP, the trajectory becomes more locally consistent (aligned segments are smoother and less jittery), which matches the low MSE trend in Fig. 4. However, ICP is still a local method and cannot fully eliminate long-term drift, so the overall corridor shape remains mildly bowed. With pose graph optimization (GTSAM), the remaining distortion is reduced further: loop-closure constraints help redistribute the accumulated error over the full trajectory, making the global shape more coherent and the long corridor closer to a straight geometry.

Seq = 21: In Fig. 3, the improvement is more visible. The dead-reckoning trajectory exhibits stronger global inconsistency, especially along the extended straight segments where drift compounds over time. ICP noticeably refines the local geometry by enforcing better scan-to-scan alignment, so the dominant path segments become tighter and less scattered, consistent with low MSE in Fig. 4. Still, a residual global deformation remains because local scan matching cannot reliably enforce loop consistency on its own. After GTSAM, the correction becomes clear: robust loop closures impose global constraints that pull the trajectory into a more self-consistent configuration, straightening the previously bowed corridor and improving the closure of the loop structure.

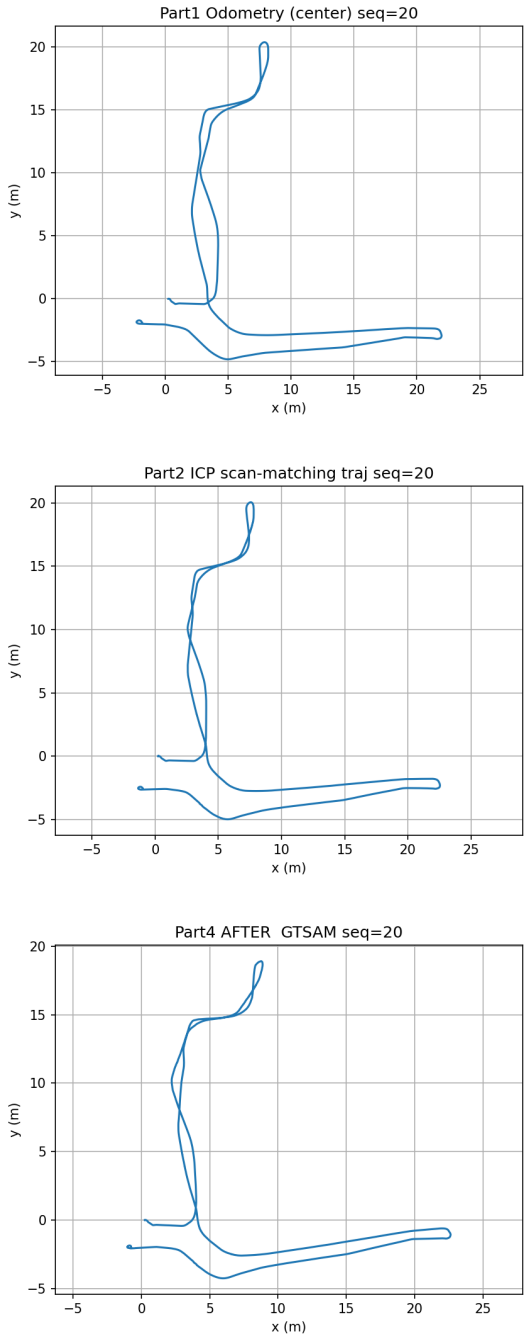


Fig. 2. Seq 20 Trajectory. Top: Dead reckoning. Middle: ICP . Bottom: GTSAM.

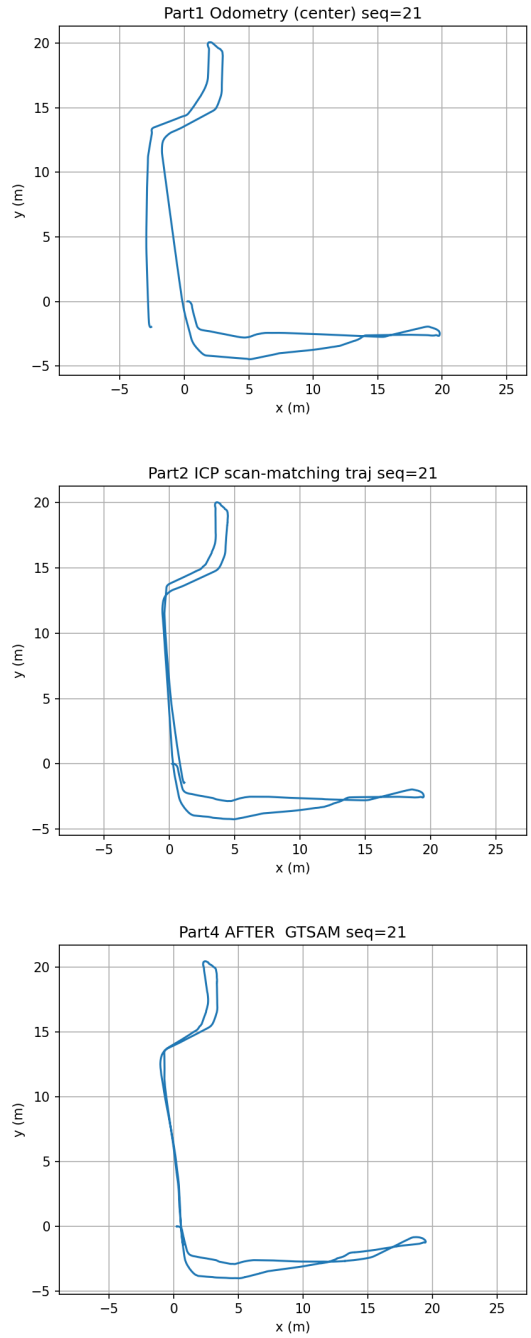


Fig. 3. Seq 21 Trajectory. Top: Dead reckoning. Middle: ICP. Bottom: GTSAM.

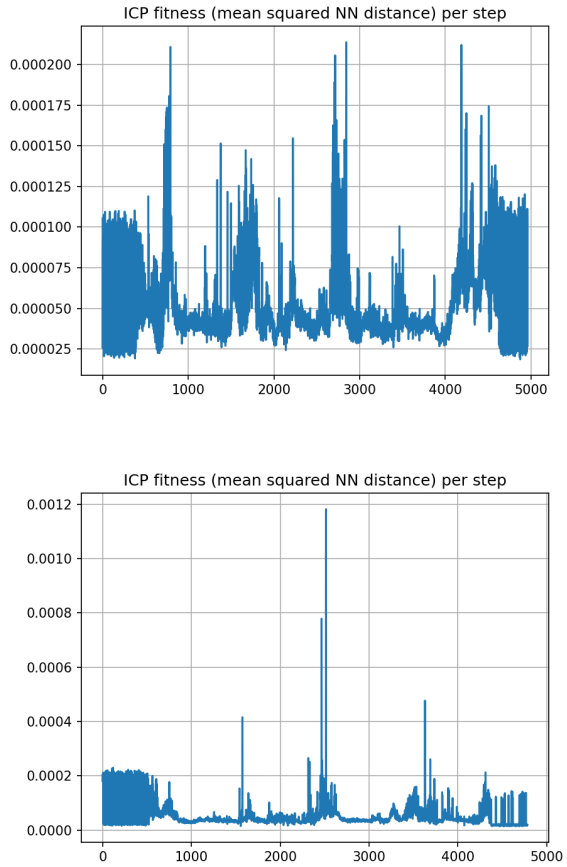


Fig. 4. ICP Mean Squared Error (MSE) fitness over time. Top: Seq 20. Bottom: Seq 21.

### C. Occupancy Grid Mapping Progression

As suggested by the project instructions, plotting the occupancy grid for just the first scan, shown in Fig. 5 and Fig. 6, Top, serves as a critical sanity check. It visually verifies that the geometric transformations and the Bresenham ray-casting logic are properly implemented before processing the entire dataset.

Regarding the full map quality, the impact of pose graph optimization is highly evident in the map boundaries. In the unoptimized maps (pure ICP), the global drift causes slight trajectory misalignments when the robot revisits an area. Consequently, the projected walls from different passes do not perfectly overlap, leading to fragmented boundaries or ”ghosting” artifacts (e.g., duplicated walls at the loop closure). After GTSAM enforces the loop closure constraints, the trajectory is precisely aligned. As a result, LiDAR rays from multiple passes perfectly superimpose onto the exact same grid cells. This repeated observation strongly reinforces the log-odds probabilities of those specific cells, turning the blurry, misaligned walls into the single, solid, and sharply defined black boundaries seen in the final maps (Fig. 5 & Fig. 6, Bottom).

Fig. 5. Occupancy Grid Maps (Seq 20). Top: First scan. Middle: Full map (pure ICP). Bottom: GTSAM optimized map.

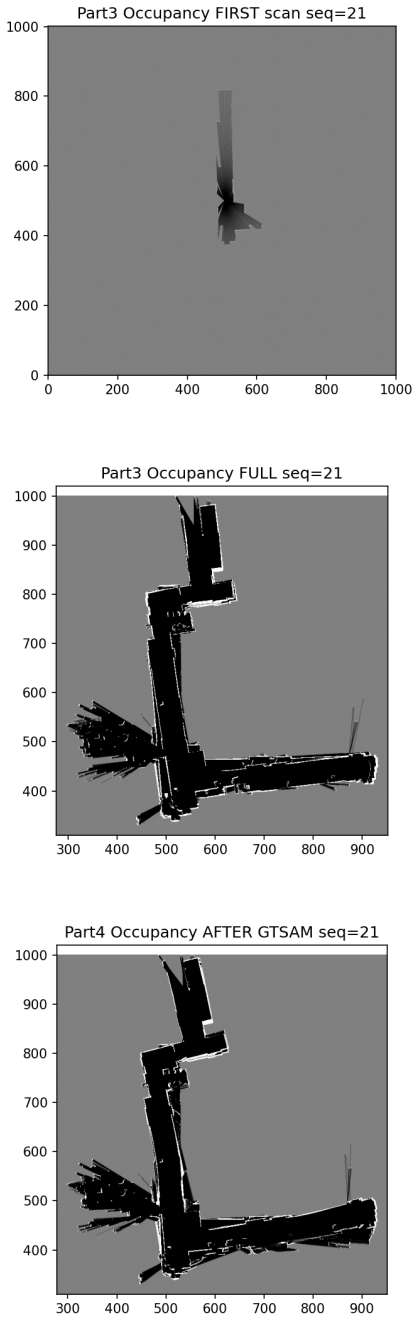


Fig. 6. Occupancy Grid Maps (Seq 21). Top: First scan. Middle: Full map (pure ICP). Bottom: GTSAM optimized map.

### D. Texture Mapping

Projecting the RGB colors onto the floor highlights the final impact of global optimization. Consistent with the occupancy maps, the overall shape of the texture maps changes only slightly after applying GTSAM, suggesting that the drift from the ICP front-end was already relatively small at the global scale. The main visible benefit of GTSAM is therefore not a dramatic shape correction, but improved consistency at boundaries and loop-closure regions. In the unoptimized texture map (Part 3), small accumulated drift can leave subtle discontinuities and gaps where revisited areas do not perfectly overlap (e.g., around the top loop region in Seq. 20). After optimization, these discontinuities are reduced and the texture boundaries become more continuous, indicating better global alignment (Fig. 7, bottom).

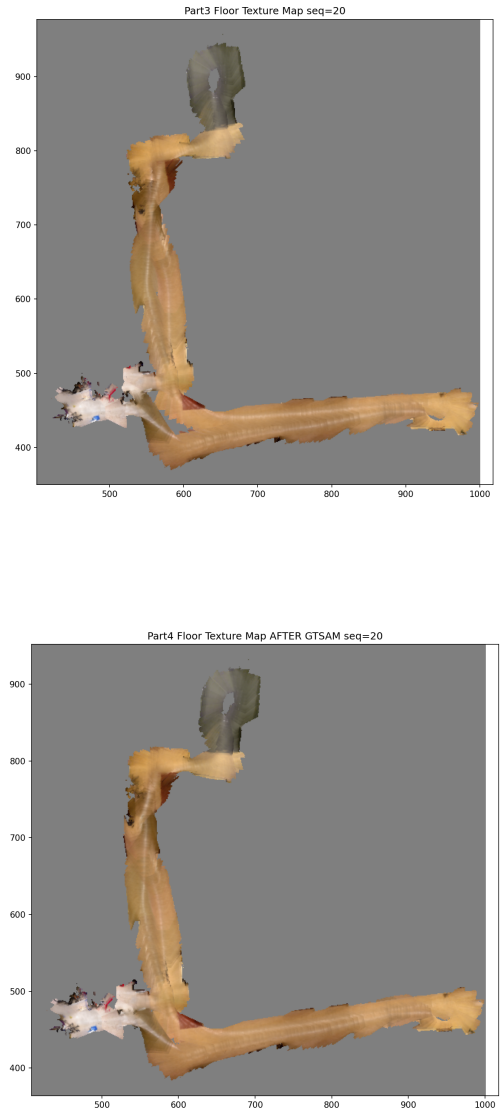


Fig. 7. Texture Maps (Seq 20). Top: Unoptimized Texture Map. Bottom: GTSAM Optimized Texture Map.

Sequence 21 is more challenging due to repeated back-and-forth traversals along long, low-texture corridors, where small misalignments can accumulate across multiple passes. In this case, tuning the pose-graph noise model becomes important: if the relative-pose uncertainty is set too loose, GTSAM may over-correct to satisfy loop closures, leading to noticeable local warping in the map. To preserve corridor rigidity, I set a small

pose uncertainty consistent with the accurate ICP front-end (e.g., $\sigma_{\text{odo}} = 0.01\,\text{m}$), so the optimization favors a rigid global adjustment rather than deforming local structure. The resulting map avoids topological artifacts, but a mild multi-pass texture thickening/blur remains, since multiple traversals are projected onto nearby but not identical pixel locations (Fig. 8).

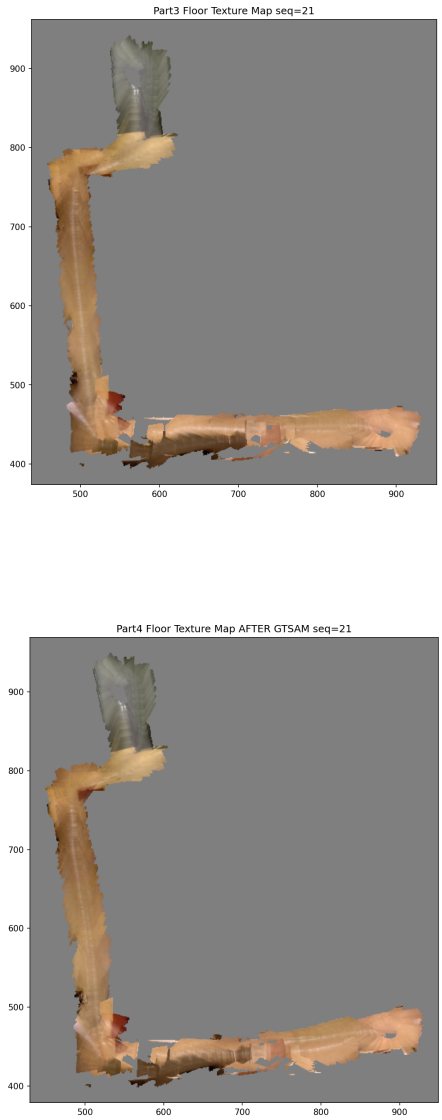


Fig. 8. Texture Maps (Seq 21). Top: Unoptimized Texture Map. Bottom: GTSAM Optimized Texture Map (structurally preserved).